Laboratório de Concorrência · Aula 10

Eduarda Pereira de Moraes · RA 24170522-2 · Turma ESOFT6S-B
Gerado em 15/10/2026, 16:17:08 · tempo registrado no cronômetro: 71:19

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Na cooperação uma tarefa precisa esperar a outra fazer uma ação para continuar. Ex: um cozinheiro espera outro preparar os ingredientes.
Na competição duas tarefas usam o mesmo recurso. Ex: duas pessoas querem usar o caixa eletrônico ao mesmo tempo.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

Resultado esperado deve ser 2.000.000, porque cada thread soma 1.000.000 no contador.

1ª execução (n = 1.000.000)

1027018

2ª execução (n = 1.000.000)

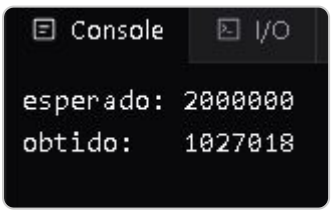
1327602

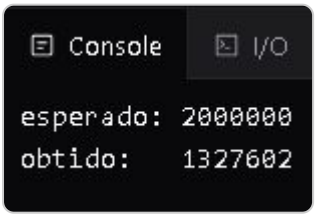
3ª execução (n = 1.000.000)

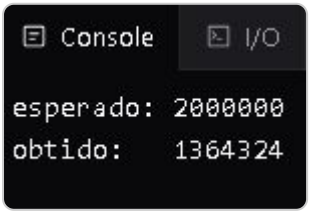
1364324

Saída do programa

```
Print:
```







1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Os valores saíram menores que o esperado, foram diferentes nas três execuções porque as duas threads acessam o mesmo contador podendo causar perda de incrementos, a ordem de execução pode mudar a cada teste.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

As duas threads podem ler o mesmo valor do contador. Cada thread soma 1 no valor lido, a primeira grava o resultado, mas a segunda também, e assim sobrescreve o valor anterior. É assim que acontece a perda de atualização.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com n=1000 todos os resultados ficaram igual ao esperado, isso mostra que a condição nem sempre causa um resultado errado. O perigo é que durante os testes o programa pode funcionar normalmente e o erro só aparecer em produção quando aumentar as operações.

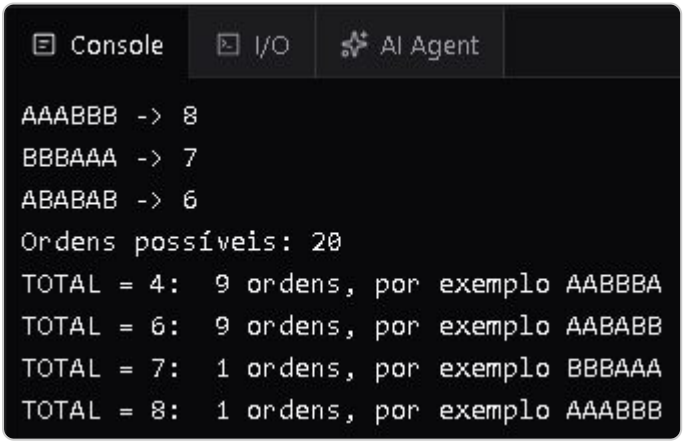
Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8
BBBAAA → 7
AABBBA → 4
BBAAAB → 6

Saída do programa

```
Print:
```



2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São possíveis 20 ordens e aparecem 4 resultados diferentes. Apenas uma ordem da o resultado correto 8.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Na ordem AABBBA a tarefa A faz a leitura do valor 3 e calcula 4, mas sem gravar. A tarefa B faz a leitura do valor 3 e multiplica por 2 e grava 6. Depois a tarefa A grava o que tinha calculado 4, sobrescrevendo o resultado da tarefa B, por isso o resultado é 4, a atualização da tarefa B se perde.

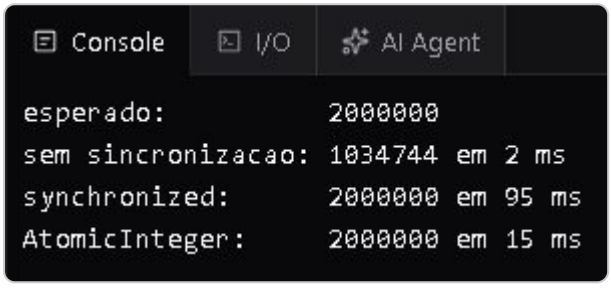
Prática 3: Corrigindo a corrida

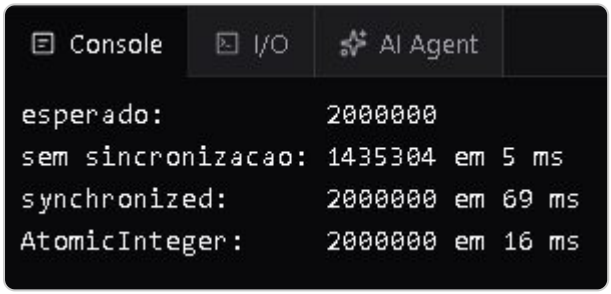
Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acho que a versão sem sincronização pode ser mais rápida, porque não controla o acesso das threads no contador. Mas ela pode gerar resultados errados, as versões synchronized e AtomicInteger devem gerar o resultado correto.

Saída do programa

```
Print:
```





3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões synchronized e atomicInteger deram o valor correto. A versão sem sincronização foi mais rápida como eu esperava, mas perdeu incrementos. Como as versões sincronizadas precisam controlar o acesso das threads no contador para evitar conflito, aumenta o tempo de execução em relação a outra versão, atomicInteger foi mais rápida que synchronized

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não, porque o atomicInteger protege cada saldo, só que não garante que a transferência inteira seja feita. Se acontecer um erro depois de tirar o dinheiro da conta o valor não pode ser incluído na outra, seria necessário sincronizar toda a operação, para que as duas sejam feitas juntas ou nenhuma seja concluída se a outra der problema.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

As mensagens não vão aparecer, porque a thread A pode ficar esperando Y, enquanto a thread B espera X, vai acontecer um impasse (deadlock)

Saídas (antes e depois da correção)

Print:





4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As duas threads ficaram presas porque a thread A estava usando X e esperando Y, enquanto a thread B estava utilizando Y e esperando X, como nenhuma liberava o recurso necessário para outra continuar ocorreu um impasse (deadlock)

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Usar os recursos sempre na mesma ordem evita impasse porque impede que as threads esperem uma pela outra . EX: sistema bancário, duas transferências entre contas podem causar impasse se cada operação bloquear uma conta diferente e esperar pela outra, ter uma ordem fixa para acessar as contas evita esse problema.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 3 trabalhadores espero que leve 200 ms, pois cada um poderá fazer duas tarefas. Com 1 trabalhador 600 ms, e com 6 trabalhadores cerca de 100 ms porque as tarefas podem ser executadas simultaneamente

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

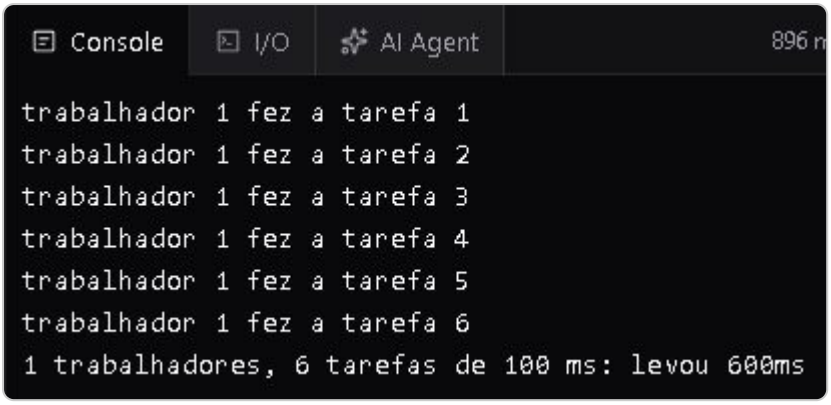
200ms

Tempo com 6 trabalhadores

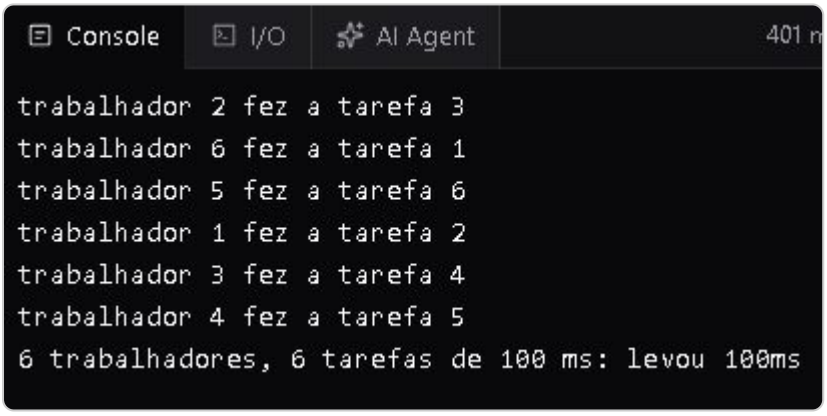
100ms

Saída do programa

Print:





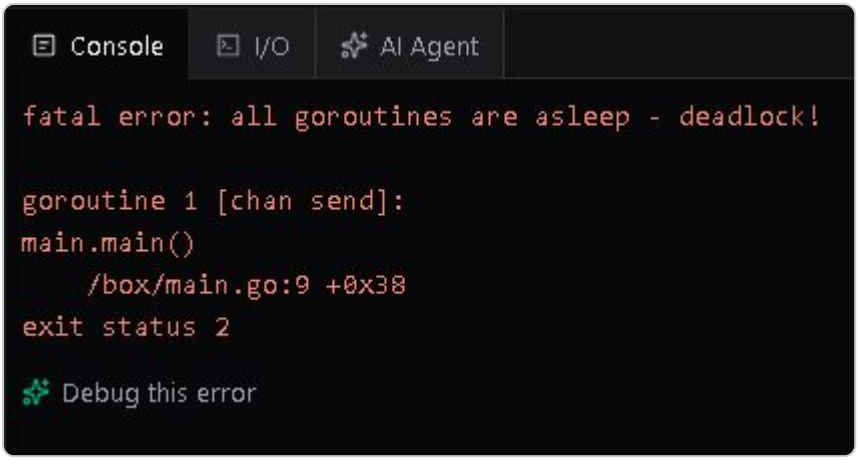


5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador as 6 tarefas foram executadas uma de cada vez levando 600 ms. Com 3 trabalhadores as tarefas foram divididas entre eles levando 200 ms. Com 6 trabalhadores todas foram executadas ao mesmo tempo levando 100 ms. O programa não precisa de synchronized nem travas porque os trabalhadores recebem as tarefas por canal sem alterar uma variável compartilhada que precisa de proteção

Saídas (antes e depois da troca)

```
Print:
```





5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro trava porque o canal não tem espaço para armazenar mensagens então o envio fica esperando alguém receber o valor mas não tem outra tarefa para isso, criando um canal com espaço para mensagem o valor pode ser armazenado sem precisar de um receptor imediato.

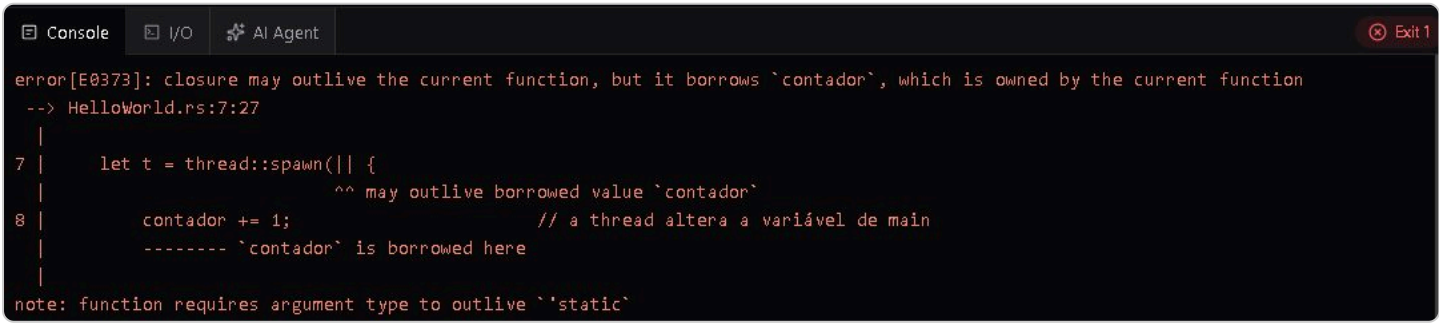
Prática 6: O compilador como guarda

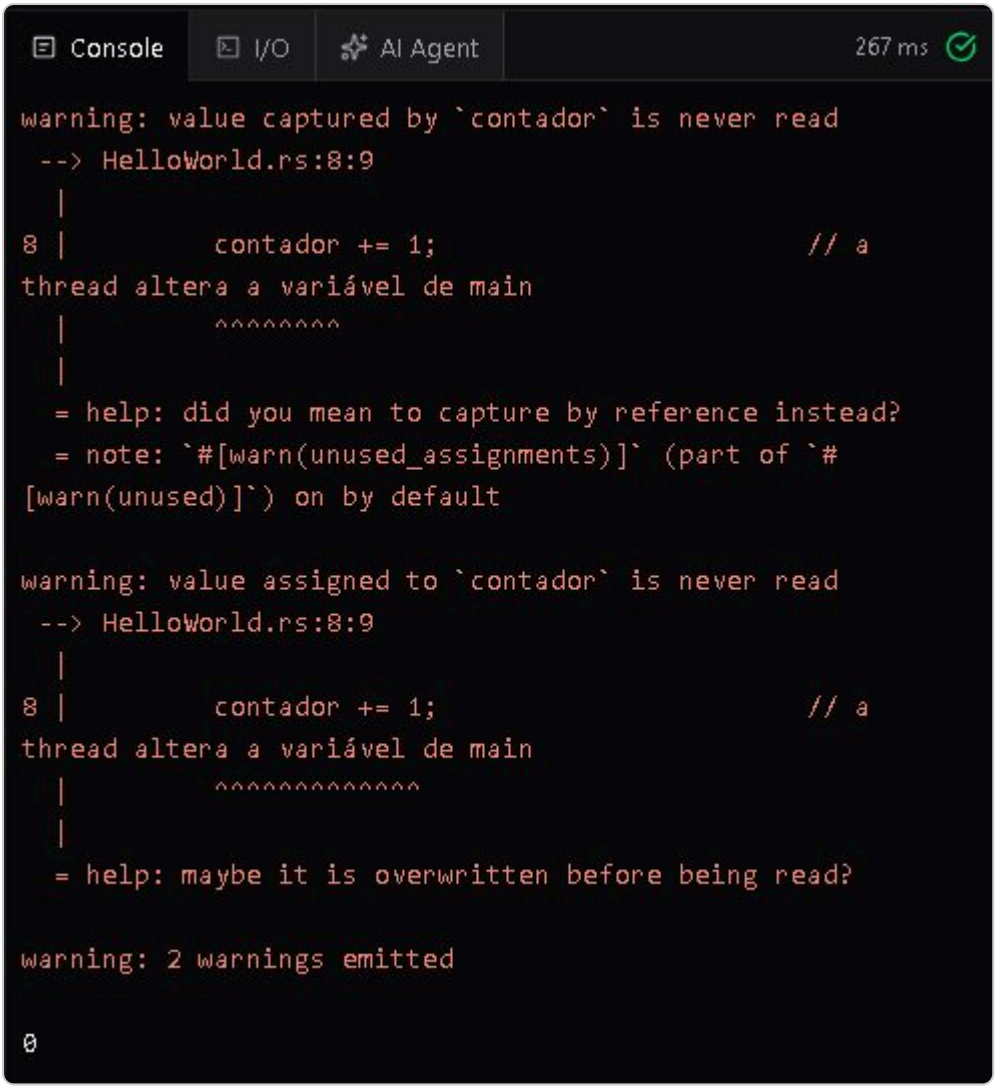
Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

O programa não vai compilar, porque a thread tenta modificar uma variável criada na main sem usar um mecanismo seguro para compartilhar.

Saídas (o erro e o resultado com move)

`Print:`





Saída do programa

```
Print:
```



6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

O primeiro não compilou porque a thread tentou acessar uma variável da main. O move o programa conseguiu compilar pois a thread recebeu uma cópia do contador. O resultado foi 0 porque a alteração aconteceu apenas nessa cópia

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java o problema foi identificado durante a execução, quando o contador apresentou valores diferentes do esperado. Em Rust o compilador identificou antes da execução. A vantagem é evitar certos erros antecipadamente mas o custo é que o programador precisa utilizar mecanismos como Arc e Mutex para compartilhar dados com segurança

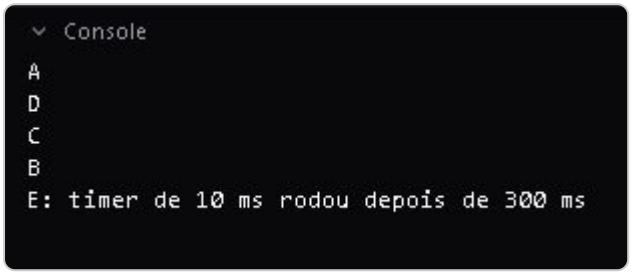
Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A ordem vai ser A, D, C, B, pois os comandos diretos executam primeiro depois apromise e por último o setTimeout.

Saída do programa

```
Print:
```



7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A, D, C e B. Porque A e D são executadas diretamente, já C é uma Promise, que tem prioridade sobre o setTimeout. Mesmo com tempo 0 B precisa esperar o código atual e as Promises terminarem para ser executada.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer de 10 ms só executou depois de 300 ms porque a função bloqueou a thread principal do JavaScript, então em um servidor Node.js um cálculo pesado pode bloquear o laço de eventos e atrasar o atendimento de outros usuários prejudicando o desempenho do sistema

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode acontecer uma condição de corrida pois as duas pessoas podem consultar o estoque ao mesmo tempo e encontrar um ingresso disponível, o mesmo lugar pode ser vendido duas vezes. Para evitar isso precisaria controlar o acesso ao estoque, garantindo apenas uma compra por vez para último ingresso.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi perceber que executar várias tarefas ao mesmo tempo não significa ter um programa mais eficiente, porque sem o controle certo podem ocorrer perda de dados, resultados incorretos e até travar o sistema.